

02-08 원-핫 인코딩(One-Hot Encoding)



컴퓨터 또는 기계는 문자보다는 숫자를 더 잘 처리 할 수 있습니다. 이를 위해 자연어 처리에서는 문자를 숫자로 바꾸는 여러가지 기법들이 있습니다. 원-핫 인코딩(One-Hot Encoding)은 그 많은 기법 중에서 단어를 표현하는 가장 기본적인 표현 방법이며, 머신러닝, 딥러닝을 하기 위해서는 반드시 배워야 하는 표현 방법입니다.

원-핫 인코딩에 대해서 배우기에 앞서 **단어 집합(vocabulary)** 에 대해서 정의해보겠습니다. 단어 집합은 앞으로 자연어 처리에서 계속 나오는 개념이기 때문에 여기서 이해하고 가야합니다. 단어 집합은 서로 다른 단어들의 집합입니다. 여기서 혼동이 없도록 서로 다른 단어라는 정의에 대해서 좀 더 주목할 필요가 있습니다. 단어 집합(vocabulary)에서는 기본적으로 book과 books와 같이 단어의 변형 형태도 다른 단어로 간주합니다. 이 책에서는 앞으로 단어 집합에 있는 단어들을 가지고, 문자를 숫자. 더 구체적으로는 벡터로 바꾸는 원-핫 인코딩을 포함한 여러 방법에 대해서 배우게 됩니다.

원-핫 인코딩을 위해서 먼저 해야할 일은 단어 집합을 만드는 일입니다. 텍스트의 모든 단어를 중복을 허용하지 않고 모아놓으면 이를 단어 집합이라고 합니다. 그리고 이 단어 집합에 고유한 정수를 부여하는 정수 인코딩을 진행합니다. 텍스트에 단어가 총 5,000개가 존재한다면, 단어 집합의 크기는 5,000입니다. 5,000개의 단어가 있는 이 단어 집합의 단어들마다 1번부터 5,000번까지 인덱스를 부여한다고 해보겠습니다. 가령, book은 150번, dog는 171번, love는 192번, books는 212번과 같이 부여할 수 있습니다.

이제 각 단어에 고유한 정수 인덱스를 부여하였다고 합시다. 이 숫자로 바뀐 단어들을 벡터로 다루고 싶다면 어떻게 하면 될까요?

1. 원-핫 인코딩(One-Hot Encoding)이란?

원-핫 인코딩은 단어 집합의 크기를 벡터의 차원으로 하고, 표현하고 싶은 단어의 인덱스에 1의 값을 부여하고, 다른 인덱스에는 0을 부여하는 단어의 벡터 표현 방식입니다. 이렇게 표현된 벡터를 원-핫 벡터(One-Hot vector)라고 합니다.

원-핫 인코딩을 두 가지 과정으로 정리해보겠습니다. 첫째, 정수 인코딩을 수행합니다. 다시 말해 각 단어에 고유한 정수를 부여합니다. 둘째, 표현하고 싶은 단어의 고유한 정수를 인덱스로 간주하고 해당 위치에 1을 부여하고, 다른 단어의 인덱스의 위치에는 0을 부여합니다. 한국어 문장을 예제로 원-핫 벡터를 만들어보겠습니다.

문장 : 나는 자연어 처리를 배운다

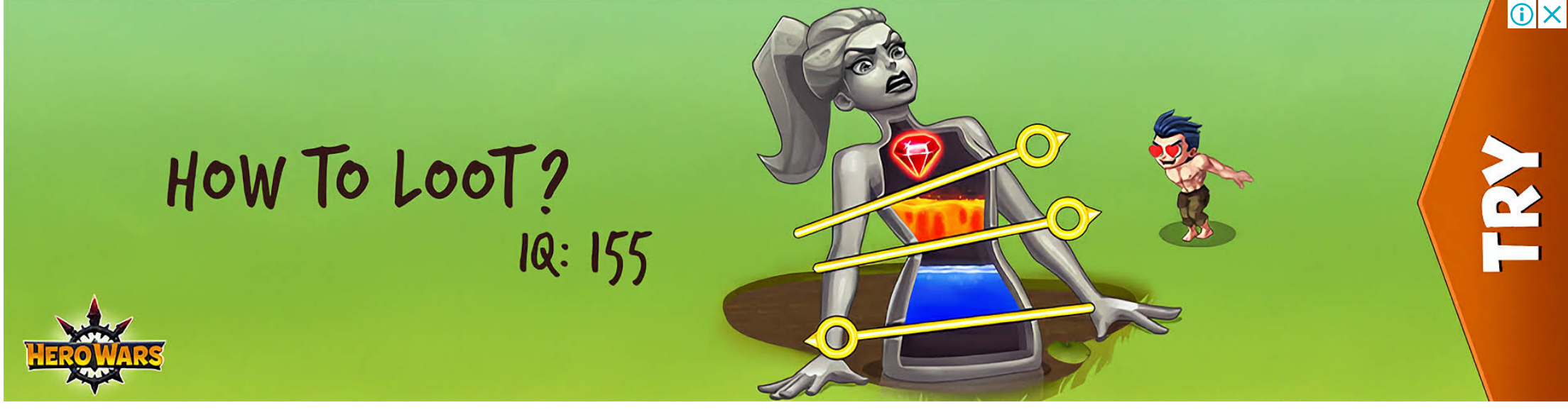
Okt 형태소 분석기를 통해서 문장에 대해서 토큰화를 수행합니다.

```
from konlpy.tag import Okt

okt = Okt()
tokens = okt.morphs("나는 자연어 처리를 배운다")
print(tokens)
```

```
['나', '는', '자연어', '처리', '를', '배운다']
```

각 토큰에 대해서 고유한 정수를 부여합니다. 지금은 문장이 짧기 때문에 각 단어의 빈도수를 고려하지 않지만, 빈도수 순으로 단어를 정렬하여 정수를 부여하는 경우가 많습니다.



```
word_to_index = {word : index for index, word in enumerate(tokens)}
print('단어 집합 : ',word_to_index)
```

```
단어 집합 : {'나': 0, '는': 1, '자연어': 2, '처리': 3, '를': 4, '배운다': 5}
```

토큰을 입력하면 해당 토큰에 대한 원-핫 벡터를 만들어내는 함수를 만들었습니다.

```
def one_hot_encoding(word, word_to_index):
    one_hot_vector = [0]*(len(word_to_index))
    index = word_to_index[word]
    one_hot_vector[index] = 1
    return one_hot_vector
```

'자연어'라는 단어의 원-핫 벡터를 얻어봅시다.

```
one_hot_encoding("자연어", word_to_index)
```

```
[0, 0, 1, 0, 0, 0]
```

'자연어'는 정수 2이므로 원-핫 벡터는 인덱스 2의 값이 1이며, 나머지 값은 0인 벡터가 나옵니다.

2. 케라스(Keras)를 이용한 원-핫 인코딩(One-Hot Encoding)

위에서는 원-핫 인코딩을 이해하기 위해 파이썬으로 직접 코드를 작성하였지만, 케라스는 원-핫 인코딩을 수행하는 유용한 도구 to_categorical()를 지원합니다. 이번에는 케라스만으로 정수 인코딩과 원-핫 인코딩을 순차적으로 진행해보도록 하겠습니다.

```
text = "나랑 점심 먹으러 갈래 점심 메뉴는 햄버거 갈래 갈래 햄버거 최고야"
```

위와 같은 문장이 있다고 했을 때, 케라스 토큰라이저를 이용한 정수 인코딩은 다음과 같습니다.

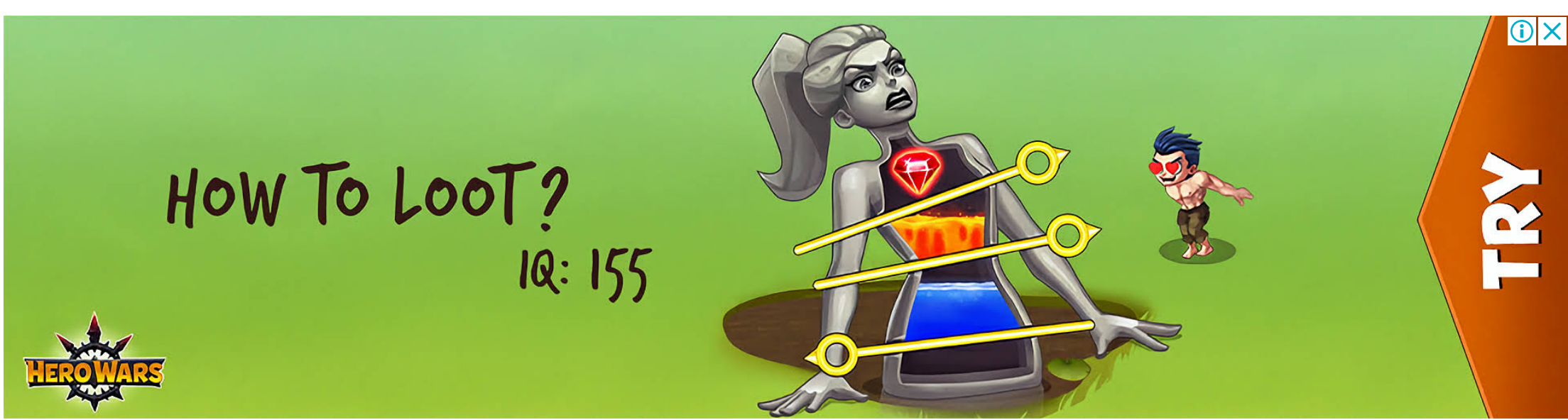
```
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.utils import to_categorical
```

```
text = "나랑 점심 먹으러 갈래 점심 메뉴는 햄버거 갈래 갈래 햄버거 최고야"
```

```
tokenizer = Tokenizer()
tokenizer.fit_on_texts([text])
print('단어 집합 : ',tokenizer.word_index)
```

```
단어 집합 : {'갈래': 1, '점심': 2, '햄버거': 3, '나랑': 4, '먹으러': 5, '메뉴는': 6, '최고야': 7}
```

위와 같이 생성된 단어 집합(vocabulary)에 있는 단어들로만 구성된 텍스트가 있다면, texts_to_sequences()를 통해서 이를 정수 시퀀스로 변환가능합니다. 생성된 단어 집합 내의 일부 단어들로만 구성된 서브 텍스트인 sub_text를 만들어 확인해보겠습니다.



```
sub_text = "점심 먹으러 갈래 메뉴는 햄버거 최고야"
encoded = tokenizer.texts_to_sequences([sub_text])[0]
print(encoded)
```

```
[2, 5, 1, 6, 3, 7]
```

지금까지 진행한 것은 이미 이전에 정수 인코딩 실습을 하며 배운 내용입니다. 이제 해당 결과를 가지고, 원-핫 인코딩을 진행해보겠습니다. 케라스는 정수 인코딩 된 결과로부터 원-핫 인코딩을 수행하는 to_categorical()를 지원합니다.

```
one_hot = to_categorical(encoded)
print(one_hot)
```

```
[[0, 0, 1, 0, 0, 0, 0, 0, 0]] # 인덱스 2의 원-핫 벡터
[0, 0, 0, 0, 0, 0, 1, 0, 0, 0]] # 인덱스 5의 원-핫 벡터
[0, 1, 0, 0, 0, 0, 0, 0, 0, 0]] # 인덱스 1의 원-핫 벡터
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0]] # 인덱스 6의 원-핫 벡터
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0]] # 인덱스 3의 원-핫 벡터
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0]] # 인덱스 7의 원-핫 벡터
```

위의 결과는 "점심 먹으러 갈래 메뉴는 햄버거 최고야"라는 문장이 [2, 5, 1, 6, 3, 7]로 정수 인코딩이 되고나서, 각각의 인코딩 된 결과를 인덱스로 원-핫 인코딩이 수행된 모습을 보여줍니다.

3. 원-핫 인코딩(One-Hot Encoding)의 한계

이러한 표현 방식은 단어의 개수가 늘어날 수록, 벡터를 저장하기 위해 필요한 공간이 계속 늘어난다는 단점이 있습니다. 다른 표현으로는 벡터의 차원이 늘어난다고 표현합니다. 원 핫 벡터는 단어 집합의 크기가 곧 벡터의 차원 수가 됩니다. 가령, 단어가 1,000개 인 코퍼스를 가지고 원 핫 벡터를 만들면, 모든 단어 각각은 모두 1,000개의 차원을 가진 벡터가 됩니다. 다시 말해 모든 단어 각각은 하나의 값만 1을 가지고, 999개의 값은 0의 값을 가지는 벡터가 되는데 이는 저장 공간 측면에서는 매우 비효율적인 표현 방법입니다.

또한 원-핫 벡터는 단어의 유사도를 표현하지 못한다는 단점이 있습니다. 예를 들어서 늑대, 호랑이, 강아지, 고양이라는 4개의 단어에 대해서 원-핫 인코딩을 해서 각각, [1, 0, 0, 0], [0, 1, 0, 0], [0, 0, 1, 0], [0, 0, 0, 1]이라는 원-핫 벡터를 부여받았다고 합시다. 이 때 원-핫 벡터로는 강아지와 늑대가 유사하고, 호랑이와 고양이가 유사하다는 것을 표현할 수가 없습니다. 좀 더 극단적으로는 강아지, 개, 냉장고라는 단어가 있을 때 강아지라는 단어가 개와 냉장고라는 단어 중 어떤 단어와 더 유사한지도 알 수 없습니다.

단어 간 유사성을 알 수 없다는 단점은 검색 시스템 등에서는 문제가 될 소지가 있습니다. 가령, 여행을 가려고 웹 검색창에 '삿포로 숙소'라는 단어를 검색한다고 합시다. 제대로 된 검색 시스템이라면, '삿포로 숙소'라는 검색어에 대해서 '삿포로 게스트 하우스', '삿포로 료칸', '삿포로 호텔'과 같은 유사 단어에 대한 결과도 함께 보여줄 수 있어야 합니다. 하지만 단어간 유사성을 계산할 수 없다면, '게스트 하우스'와 '료칸'과 '호텔'이라는 연관 검색어를 보여줄 수 없습니다.

이러한 단점을 해결하기 위해 단어의 잠재 의미를 반영하여 다차원 공간에 벡터화 하는 기법으로 크게 두 가지가 있습니다. 첫째는 카운트 기반의 벡터화 방법인 LSA(잠재 의미 분석), HAL 등이 있으며, 둘째는 예측 기반으로 벡터화하는 NNLM, RNNLM, Word2Vec, FastText 등이 있습니다. 그리고 카운트 기반과 예측 기반 두 가지 방법을 모두 사용하는 방법으로 GloVe라는 방법이 존재합니다.

여기서 언급한 방법들 중 대부분은 워드 임베딩 챕터에서 다루게 됩니다.

마지막 편집일시 : 2022년 11월 14일 2:43 오후